



Secure FPGA-Embedded Blockchain with MKRAND and Nash Cipher

July 17, 2025

Abstract

This document outlines a proposal for co-developing a secure, FPGA-based blockchain capability using MKRAND (a post-quantum entropy generator) and the Nash Cipher (a lightweight stream encryption engine). These IP blocks are designed for drop-in use across Enclustra's SoC module portfolio, enabling immediate value in secure communications, trusted timestamping, and offline transaction settlement.

1 Executive Summary

Enclustra's SoC modules are widely deployed across industrial, financial, and defense markets. By integrating MKRAND and Nash Cipher, these platforms can offer:

- Deterministic cryptographic timestamping at the hardware level
- Lightweight, unbreakable data streams without key exchange
- Secure, offline-capable blockchain infrastructure for industrial edge, POS, and military

This is a rare opportunity to integrate next-gen cryptography into existing product lines with minimal effort.

Strategic Licensing Model (Preliminary Proposal)

To ensure rapid deployment and low friction for integration, the MKRAND and Nash Cipher IP cores are being made available to Enclustra under an open evaluation framework. No licensing fees or royalties are required during the initial development and integration phase.

Once integration is successful and the technology is ready for commercial deployment, we propose entering into a formal licensing agreement structured around one or more of the following models:

- **Royalty per Module:** A nominal per-unit fee on each Enclustra SoC or FPGA board embedding MKRAND/Nash, with pricing scaled to volume and tier of customer.

- **Flat Licensing Fee:** A single up-front fee for unlimited internal use, with optional support and update agreements.
- **Exclusive Rights Package:** A limited-time exclusivity window (e.g., 6–12 months) where Enclustra is the sole distributor of MKRAND/Nash-embedded hardware, in exchange for a negotiated exclusivity payment.

Why This Matters

- Enclustra becomes the first-to-market vendor offering **offline-capable, silicon-native blockchain and post-quantum encryption** — a key differentiator in aerospace, defense, and critical infrastructure.
- Minimal integration effort due to RTL simplicity and AXI-compatible interface.
- Licensing can remain non-binding until commercial rollout — allowing your engineering team to fully evaluate the IP without pressure.

This model ensures zero friction today, while paving the way for future monetization aligned with deployment success.

2 System Overview

Dataflow Summary

- MKRAND generates 128-bit monotonic time-stamps
- Nash Cipher encrypts data with forward secrecy
- Blockchain block emitted per transaction and stored locally or broadcast via secure link

3 Use Case Scenarios

- **Central Bank Digital Currency (CBDC):** Hardware-level offline settlements
- **Military Communication Modules:** Self-contained mesh nodes with zero-trust encryption
- **Retail / POS Systems:** Tokenized receipts and payment logs on immutable hardware chain
- **Industrial IoT:** Machine logbooks secured cryptographically at the source

4 Technical Architecture

Integration Requirements

- Compatible with standard AXI/Lite bus or memory-mapped registers
- Requires access to a 64–128 MHz clock source and reset
- Generates one 128-bit block per 256 cycles at full speed

5 Core IP Descriptions

5.1 MKRAND: Monotonic Timestamp Generator

- Post-quantum deterministic random bit generator
- Generates high-entropy time vectors
- Can be fused into a VHDL/Verilog module with zero external dependencies

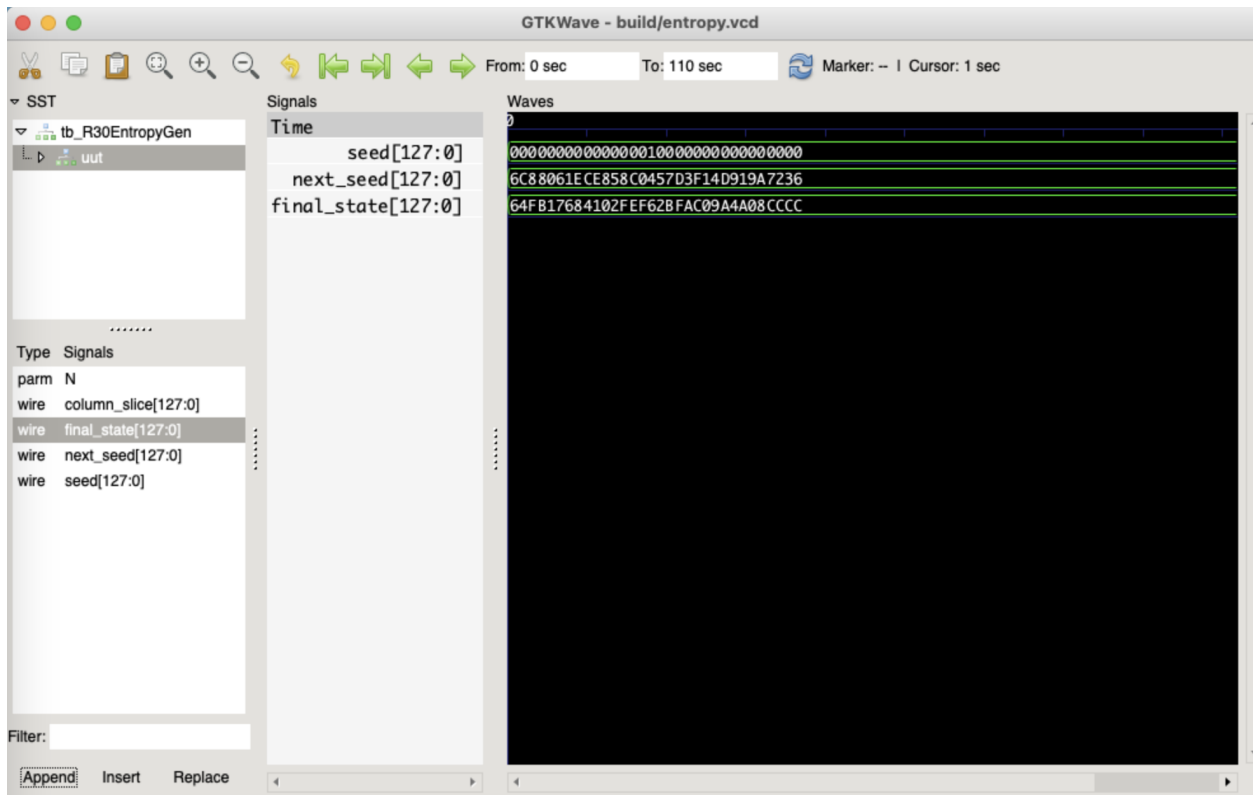


Figure 1: One-shot entropy extraction using Rule 30.

The seed (top) evolves through 256 iterations of Rule 30, producing a final state (bottom). The entropy output (`next_seed`, middle) is derived by XOR'ing the lower half of the center column with the original seed.

5.2 Nash Cipher: Lightweight Stream Encryption

- No nonce, no IV, no key negotiation required
- Stream cipher optimized for FPGA implementation
- Latency: 256 cycles per block

Encryption Example:

Input: PLAINTEXT_BLOCK

Key: MKRAND_OUTPUT

Output: CIPHERTEXT_BLOCK

6 Integration Roadmap

1. **Week 1–2:** NDA and technical handoff of MKRAND/Nash IP cores
2. **Week 3–4:** Enclustra firmware team integrates into evaluation board
3. **Month 2:** Internal pilot use for payment or defense module
4. **Month 3+:** External rollout and licensing discussions

7 Appendices

Appendix A: Sample VHDL Interface

```
1 entity mkrand_core is
2   Port (
3     clk          : in  std_logic;
4     reset        : in  std_logic;
5     seed_vector  : in  std_logic_vector(127 downto 0);
6     out_vector   : out std_logic_vector(127 downto 0)
7   );
8 end mkrand_core;
```

R30Cell Module

```
1 module R30Cell (
2   input wire L,
3   input wire C,
4   input wire R,
5   output wire R30CellOut
6 );
7
8 assign R30CellOut = L ^ (C | R);
9
```

```
10 endmodule
```

R30Array Module

```
1 module R30Array #(
2     parameter N = 128
3 ) (
4     input wire [N-1:0] state_in,
5     output wire [N-1:0] state_out
6 );
7
8     wire [N-1:0] cell_outputs;
9
10    genvar i;
11    generate
12        for (i = 0; i < N; i = i + 1) begin : cell_array
13            wire L_i, C_i, R_i;
14
15            assign L_i = state_in[(i == 0) ? N-1 : i-1];
16            assign C_i = state_in[i];
17            assign R_i = state_in[(i == N-1) ? 0 : i+1];
18
19            R30Cell cell_inst (
20                .L(L_i),
21                .C(C_i),
22                .R(R_i),
23                .R30CellOut(cell_outputs[i])
24            );
25        end
26    endgenerate
27
28    assign state_out = cell_outputs;
29
30 endmodule
```

R30Field Module

```
1 module R30Field #(
2     parameter N = 128,
3     parameter D = 256
4 ) (
5     input wire [N-1:0] seed,
6     output wire [N-1:0] final_state,
7     output wire [127:0] column_slice
8 );
9
```

```

10     wire [N-1:0] states [0:D];
11     assign states[0] = seed;
12
13     genvar i;
14     generate
15         for (i = 0; i < D; i = i + 1) begin : r30_steps
16             R30Array #(.N(N)) step (
17                 .state_in(states[i]),
18                 .state_out(states[i+1])
19             );
20         end
21     endgenerate
22
23     assign final_state = states[D];
24
25     genvar j;
26     generate
27         for (j = 0; j < 128; j = j + 1) begin : extract_column
28             assign column_slice[j] = states[128 + j][N/2];
29         end
30     endgenerate
31
32 endmodule

```

R30EntropyGen Module

```

1 module R30EntropyGen #(
2     parameter N = 128
3 ) (
4     input  wire [N-1:0] seed,
5     output wire [N-1:0] next_seed
6 );
7
8     wire [N-1:0] final_state;
9     wire [N-1:0] column_slice;
10
11     R30Field #(.N(N), .D(256)) field (
12         .seed(seed),
13         .final_state(final_state),
14         .column_slice(column_slice)
15     );
16
17     assign next_seed = seed ^ column_slice;
18
19 endmodule

```

Appendix B: NashCipher Design Sketch (Verilog)

```
1 module NashCipher (  
2     input wire clk,  
3     input wire reset,  
4     input wire input_bit,  
5     input wire decider_bit,  
6     output reg output_bit  
7 );  
8  
9     // State and memory arrays  
10    rreg [6:0] current_state = 0; // 7 bits to represent 0-127  
11  
12    reg [6:0] red_permuter [0:127];  
13    reg [6:0] blue_permuter [0:127];  
14    reg flip_matrix [0:127];  
15  
16    integer i;  
17  
18    // Optional: initialize the permuters and matrix (can be  
19    // overridden externally)  
20    initial begin  
21        for (i = 0; i < 128; i = i + 1) begin  
22            red_permuter[i] = i; // Identity by default  
23            blue_permuter[i] = i;  
24            flip_matrix[i] = 0; // No flipping by default  
25        end  
26    end  
27  
28    always @(posedge clk or posedge reset) begin  
29        if (reset) begin  
30            current_state <= 0;  
31        end else begin  
32            // Update state based on decider bit  
33            if (decider_bit == 1'b0)  
34                current_state <= red_permuter[current_state];  
35            else  
36                current_state <= blue_permuter[current_state];  
37  
38            // Flip or pass through  
39            if (flip_matrix[current_state])  
40                output_bit <= ~input_bit;  
41            else  
42                output_bit <= input_bit;  
43        end  
44    end  
45 endmodule
```


Nash Cipher Integration Note

The **Nash Cipher** is a lightweight stream cipher optimized for hardware implementation, using deterministic state transitions based on a pair of internal permuters and conditional logic. It requires minimal resources and is scalable beyond its default 128-bit configuration.

Key Generation Requirements

To initialize the cipher securely, the following components are required:

1. **Two independently shuffled permutations** of integers from 0 to 127:
 - `red_permuter[0..127]`
 - `blue_permuter[0..127]`
2. **Two randomized flip matrices**, each of 128 bits:
 - One for use when traversing the red permutation path
 - One for the blue path

Each flip matrix defines whether the input bit should be flipped (1) or passed through unchanged (0) at a given state.

Note: All four of these structures must be generated with cryptographic randomness to preserve forward secrecy and resist reconstruction. The MKRAND core is an ideal entropy source for this purpose.

MKRAND as a Key Generator

The MKRAND entropy engine can be used to generate:

- Permuted sequences via random sampling and shuffling algorithms
- Flip matrices by extracting random bits

Each configuration of the Nash Cipher becomes a unique stream cipher instance when seeded independently.

MKRAND as a Standalone Cipher

In addition to its role as a key generator, MKRAND may also be used directly as an encryption engine:

- Given an initial 128-bit seed, MKRAND produces a deterministic, high-entropy sequence of blocks.
- Each output block can be XORed with a corresponding block of plaintext to yield ciphertext.

- Decryption uses the same seed and produces an identical stream, enabling symmetric decoding.

This method allows for practically infinite encrypted streams, limited only by hardware or bandwidth constraints.

Scalability

Both MKRAND and Nash Cipher are scalable:

- The internal width (default 128 bits) can be extended to 256, 512, or higher.
- The cipher mechanism remains unchanged—only the array dimensions and transition depth increase.
- No architectural redesign is required to scale to longer bitstreams or larger state spaces.

This modularity makes the Nash Cipher and MKRAND especially suitable for:

- Embedded hardware constraints
- High-performance applications
- Systems requiring post-quantum and offline encryption guarantees

Appendix B: Patent Reference

- Patent application describing the architecture for a Central Bank Digital Currency (CBDC) that works offline using printable blockchain blocks. [Application 63/800,999].
- Patent application for MKRAND, a cryptographically secure random bit generator designed for deterministic event ordering across isolated blockchain nodes. [Application 63/799,923].
- Patent application introducing the Nash Cipher, a novel encryption system designed for ultra-lightweight, post-quantum streaming encryption. [Application 63/800,566]

Appendix C: Contact

Enrique Flores

taguniversalsoftware@gmail.com

Patent docs: https://github.com/taguniversal/digital_blockchain_patents

Verilog sketches in src/rtl directory: <https://github.com/taguniversal/rc>